8
Advanced Object Detection

In the previous chapter, we learned about the R-CNN and Fast R-CNN techniques, which leverage region proposals to generate predictions of the locations of objects in an image along with the classes corresponding to objects in the image. Furthermore, we learned about the bottleneck of the speed of inference, which happens due to having two different models – one for region proposal generation and another for object detection. In this chapter, we will learn about different modern techniques, such as Faster R-CNN, YOLO, and **single-shot detector** (**SSD**), that overcome slow inference time by employing a single model to make predictions for both the class of the object and the bounding box in a single shot. We will start by learning about anchor boxes and then proceed to learn how each of the techniques works and how to implement them to detect objects in an image.

We will cover the following topics in this chapter:

- Components of modern object detection algorithms
- Training Faster R-CNN on a custom dataset
- Working details of YOLO
- Training YOLO on a custom dataset
- Working details of SSD
- Training SSD on a custom dataset

In addition to the above, as a bonus, we have covered the following in the GitHub repository:

- Training YOLOv8
- Training the EfficientDet architecture

All code snippets within this chapter are available in the `Chapter08` folder of the GitHub repository at https://bit.ly/mcvp-2e.

As the field evolves, we will periodically add valuable supplements to the GitHub repository. Do check the `supplementary_sections` folder within each chapter's directory for new and useful content.

Components of modern object detection algorithms

The drawback of the R-CNN and Fast R-CNN techniques is that they have two disjointed networks – one to identify the regions that likely contain an object and the other to make corrections to the bounding box where an object is identified. Furthermore, both models require as many forward propagations as there are region proposals. Modern object detection algorithms focus heavily on training a single neural network and have the capability to detect all objects in one forward pass. The various components of a typical modern object detection algorithm are:

- Anchor boxes
- Region proposal network (RPN)
- Region of interest (RoI) pooling

Let's discuss these in the following subsections (we'll be focusing on anchor boxes and RPN as we discussed RoI pooling in the previous chapter).

Anchor boxes

So far, we have had region proposals coming from the `selectivesearch` method. Anchor boxes come in as a handy replacement for selective search – we will learn how they replace `selectivesearch`-based region proposals in this section.

Typically, a majority of objects have a similar shape – for example, in a majority of cases, a bounding box corresponding to an image of a person will have a greater height than width, and a bounding box corresponding to the image of a truck will have a greater width than height. Thus, we will have a decent idea of the height and width of the objects present in an image even before training the model (by inspecting the ground truths of bounding boxes corresponding to objects of various classes).

Furthermore, in some images, the objects of interest might be scaled – resulting in a much smaller or much greater height and width than average – while still maintaining the aspect ratio (that is, height/weight).

Once we have a decent idea of the aspect ratio and the height and width of objects (which can be obtained from ground-truth values in the dataset) present in our images, we define the anchor boxes with heights and widths representing the majority of objects' bounding boxes within our dataset. Typically, this is obtained by employing K-means clustering on top of the ground-truth bounding boxes of objects present in images.

Now that we understand how anchor boxes' heights and widths are obtained, we will learn how to leverage them in the process:

1. Slide each anchor box over an image from top left to bottom right.
2. The anchor box that has a high **intersection over union** (**IoU**) with the object will have a label that mentions that it contains an object, and the others will be labeled `0`.

We can modify the threshold of the IoU by mentioning that if the IoU is greater than a certain threshold, the object class is `1`; if it is less than another threshold, the object class is `0`, and it is unknown otherwise.

Once we obtain the ground truths as defined here, we can build a model that can predict the location of an object and also the offset corresponding to the anchor box to match it with the ground truth. Let's now understand how anchor boxes are represented in the following image:

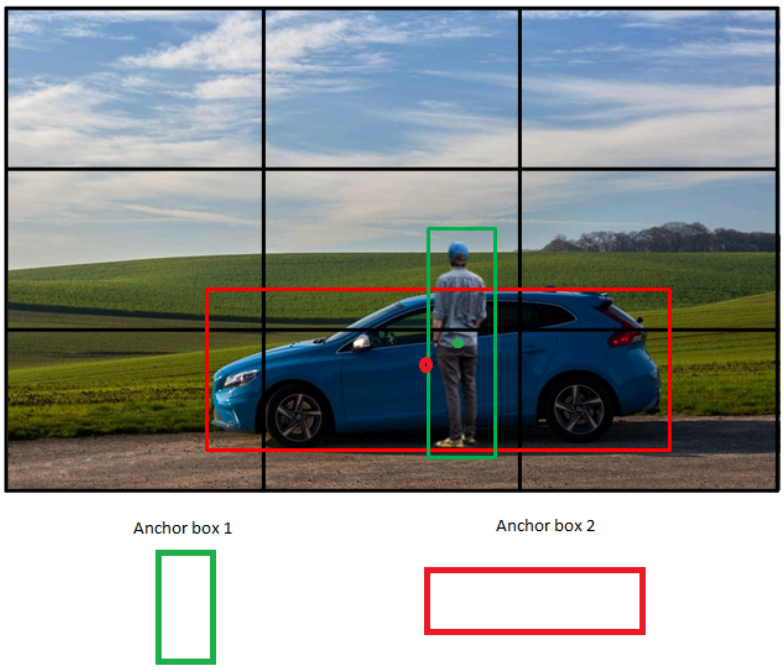

Figure 8.1: Sample anchor boxes

In the preceding image, we have two anchor boxes, one that has a greater height than width and the other with a greater width than height, to correspond to the objects (classes) in the image – a person and a car.

We slide the two anchor boxes over the image and note the locations where the IoU of the anchor box with the ground truth is the highest and denote that this particular location contains an object while the rest of the locations do not contain an object.

In addition to the preceding two anchor boxes, we would also create anchor boxes with varying scales so that we accommodate the differing scales at which an object can be presented within an image. An example of how the different scales of anchor boxes is as follows. Note that all the anchor boxes have the same center but different aspect ratios or scales:

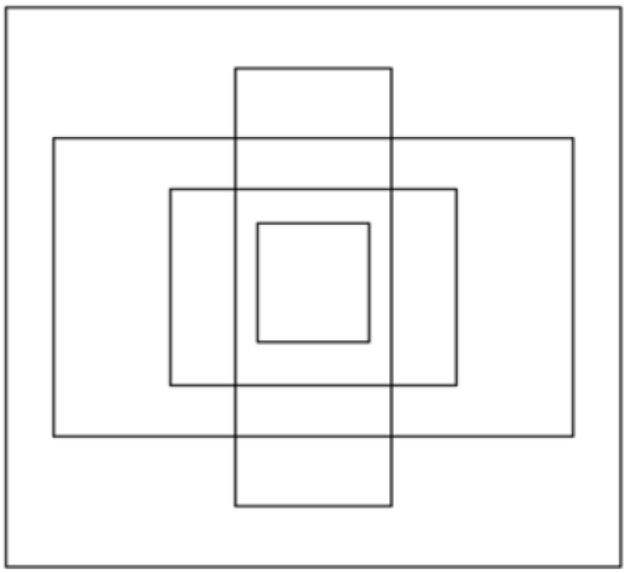

Figure 8.2: Anchor boxes with different scale and aspect ratios

Now that we understand anchor boxes, in the next section, we will learn about the RPN, which leverages anchor boxes to come up with predictions of regions that are likely to contain an object.

Region proposal network

Imagine a scenario where we have a 224 x 224 x 3 image. Furthermore, let's say that the anchor box is of shape 8 x 8 for this example. If we have a stride of 8 pixels, we are fetching 224/8 = 28 crops of a picture for every row – essentially 28*28 = 576 crops from a picture. We then take each of these crops and pass them through an RPN that indicates whether the crop contains an object. Essentially, an RPN suggests the likelihood of a crop containing an object.

Let's compare the output of `selectivesearch` and the output of an RPN.

`selectivesearch` gives us a region candidate based on a set of computations on top of pixel values. However, an RPN generates region candidates based on the anchor boxes and the strides with which anchor boxes are slid over the image. Once we obtain the region candidates using either of these two methods, we identify the candidates that are most likely to contain an object.

While region proposal generation based on `selectivesearch` is done outside of the neural network, we can build an RPN that is a part of the object detection network. Using an RPN, we are now in a position where we don't have to perform unnecessary computations to calculate region proposals outside of the network. This way, we have a single model to identify regions, identify classes of objects in an image, and identify their corresponding bounding box locations.

Next, we will learn how an RPN identifies whether a region candidate (a crop obtained after sliding an anchor box) contains an object or not. In our training data, we would have the ground truth correspond to objects. We now take each region candidate and compare it with the ground-truth bounding boxes of objects in an image to identify whether the IoU between a region candidate and a ground-truth bounding box is greater than a certain threshold. If the IoU is greater than a certain threshold (say, `0.5`), the region candidate contains an object, and if the IoU is less than a threshold (say, `0.1`), the region candidate does not contain an object and all the candidates that have an IoU between the two thresholds (`0.1` and `0.5`) are ignored while training.

Once we train a model to predict if the region candidate contains an object, we then perform non-max suppression, as multiple overlapping regions can contain an object.

In summary, an RPN trains a model to enable it to identify region proposals with a high likelihood of containing an object by performing the following steps:

1. Slide anchor boxes of different aspect ratios and sizes across the image to fetch crops of an image.
2. Calculate the IoU between the ground-truth bounding boxes of objects in the image and the crops obtained in the previous step.
3. Prepare the training dataset in such a way that crops with an IoU greater than a threshold contain an object and crops with an IoU less than a threshold do not contain an object.
4. Train the model to identify regions that contain an object.
5. Perform non-max suppression to identify the region candidate that has the highest probability of containing an object and eliminate other region candidates that have a high overlap with it.

We now pass the region candidates through an RoI pooling layer to get regions of the shape.

Classification and regression

So far, we have learned about the following steps in order to identify objects and perform offsets to bounding boxes:

1. Identify the regions that contain objects.
2. Ensure that all the feature maps of regions, irrespective of the regions' shape, are exactly the same using RoI pooling (which we learned about in the previous chapter).

Two issues with these steps are as follows:

- The region proposals do not correspond tightly over the object (`IoU>0.5` is the threshold we had in the RPN).
- We identified whether the region contains an object or not, but not the class of the object located in the region.

We address these two issues in this section, where we take the uniformly shaped feature map obtained previously and pass it through a network. We expect the network to predict the class of the object contained within the region and also the offsets corresponding to the region to ensure that the bounding box is as tight as possible around the object in the image.

Let's understand this through the following diagram:

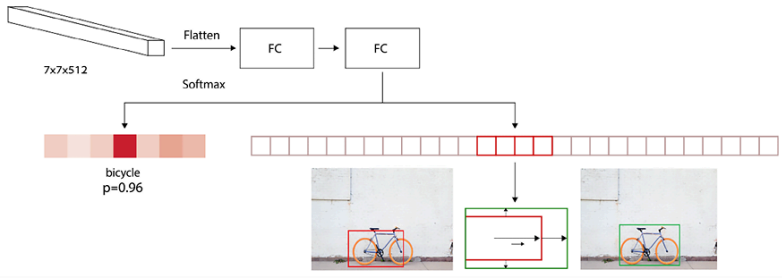

Figure 8.3: Predicting the class of object and the offset to be done to the predicted bounding box

In the preceding diagram, we are taking the output of RoI pooling as input (the 7 x 7 x 512 shape), flattening it, and connecting it to a dense layer before predicting two different aspects:

- Class of object in the region
- Amount of offset to be done on the predicted bounding boxes of the region to maximize the IoU with the ground truth

Hence, if there are 20 classes in the data, the output of the neural network contains a total of 25 outputs – 21 classes (including the background class) and the 4 offsets to be applied to the height, width, and two center coordinates of the bounding box.

Now that we have learned about the different components of an object detection pipeline, let's summarize it with the following diagram:

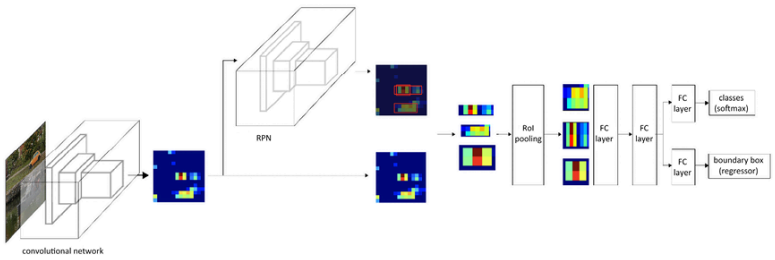

Figure 8.4: Faster R-CNN workflow

More details about Faster R-CNN can be found in the paper here – **https://arxiv.org/pdf/1506.01497.pdf**.

With the working details of each of the components of Faster R-CNN in place, in the next section, we will code up object detection using the Faster R-CNN algorithm.

Training Faster R-CNN on a custom dataset

In the following code, we will train the Faster R-CNN algorithm to detect the bounding boxes around objects present in images. For this, we will work on the same truck versus bus detection exercise from the previous chapter:

Find the following code in the `Training_Faster_RCNN.ipynb` file in the `Chapter08` folder on GitHub at **https://bit.ly/mcvp-2e**.

1. Download the dataset:

```
!pip install -qU torch_snippets
import os
%%writefile kaggle.json
 {"username":"XXX", "key":"XXX"}
 !mkdir -p ~/.kaggle
 !cp kaggle.json ~/.kaggle/
 !chmod 600 /root/.kaggle/kaggle.json
 !kaggle datasets download -d sixhky/open-images-bus-trucks/
 !unzip -qq open-images-bus-trucks.zip
 !rm open-images-bus-trucks.zip
```

2. Read the DataFrame containing metadata of information about images and their bounding box, and classes:

```
from torch_snippets import *
from PIL import Image
IMAGE_ROOT = 'images/images'
DF_RAW = df = pd.read_csv('df.csv')
```

3. Define the indices corresponding to labels and targets:

```
label2target = {l:t+1 for t,l in enumerate(DF_RAW['LabelName'].unique())}
label2target['background'] = 0
target2label = {t:l for l,t in label2target.items()}
background_class = label2target['background']
num_classes = len(label2target)
```

4. Define the function to preprocess an image – `preprocess_image`:

```
def preprocess_image(img):
    img = torch.tensor(img).permute(2,0,1)
    return img.to(device).float()
```

5. Define the dataset class – `OpenDataset`:
    1. Define an `__init__` method that takes the folder containing images and the DataFrame containing the metadata of the images as inputs:

```
class OpenDataset(torch.utils.data.Dataset):
    w, h = 224, 224
    def __init__(self, df, image_dir=IMAGE_ROOT):
        self.image_dir = image_dir
        self.files = glob.glob(self.image_dir+'/*')
        self.df = df
        self.image_infos = df.ImageID.unique()
```

    2. Define the `__getitem__` method, where we return the preprocessed image and the target values:

```
    def __getitem__(self, ix):
        # load images and masks
        image_id = self.image_infos[ix]
        img_path = find(image_id, self.files)
        img = Image.open(img_path).convert("RGB")
        img = np.array(img.resize((self.w, self.h),
                                  resample=Image.BILINEAR))/255.
        data = df[df['ImageID'] == image_id]
        labels = data['LabelName'].values.tolist()
        data = data[['XMin','YMin','XMax','YMax']].values
        # Convert to absolute coordinates
        data[:,[0,2]] *= self.w
        data[:,[1,3]] *= self.h
        boxes = data.astype(np.uint32).tolist()
        # torch FRCNN expects ground truths as
        # a dictionary of tensors
        target = {}
        target["boxes"] = torch.Tensor(boxes).float()
        target["labels"]= torch.Tensor([label2target[i] \
                                  for i in labels]).long()
        img = preprocess_image(img)
        return img, target
```

Note that, for the first time, we are returning the output as a dictionary of tensors and not as a list of tensors. This is because the official PyTorch implementation of the `FRCNN` class expects the target to contain the absolute coordinates of bounding boxes and the label information.

    3. Define the `collate_fn` method (by default, `collate_fn` works only with tensors as inputs, but here, we are dealing with a list of dictionaries) and the `__len__` method:

```
    def collate_fn(self, batch):
        return tuple(zip(*batch))
    def __len__(self):
        return len(self.image_infos)
```

6. Create the training and validation dataloaders and datasets:

```
from sklearn.model_selection import train_test_split
trn_ids, val_ids = train_test_split(df.ImageID.unique(),
                    test_size=0.1, random_state=99)
trn_df, val_df = df[df['ImageID'].isin(trn_ids)], \
                    df[df['ImageID'].isin(val_ids)]
train_ds = OpenDataset(trn_df)
test_ds = OpenDataset(val_df)
train_loader = DataLoader(train_ds, batch_size=4,
                          collate_fn=train_ds.collate_fn,
                                        drop_last=True)
test_loader = DataLoader(test_ds, batch_size=4,
                           collate_fn=test_ds.collate_fn,
                                        drop_last=True)
```

7. Define the model:

```
import torchvision
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor
device = 'cuda' if torch.cuda.is_available() else 'cpu'
def get_model():
    model = torchvision.models.detection\
                .fasterrcnn_resnet50_fpn(pretrained=True)
    in_features = model.roi_heads.box_predictor.cls_score.in_features
    model.roi_heads.box_predictor = \
                 FastRCNNPredictor(in_features, num_classes)
    return model
```

The model contains the following key submodules:

```
==========================================================================
Layer (type:depth-idx)                   Param #
==========================================================================
├─GeneralizedRCNNTransform: 1-1          --
├─BackboneWithFPN: 1-2                   (26,799,296)
├─RegionProposalNetwork: 1-3             593,935
├─RoIHeads: 1-4                          13,905,930
==========================================================================
Total params: 41,299,161
Trainable params: 14,499,865
Non-trainable params: 26,799,296
==========================================================================
```

Figure 8.5: Faster R-CNN architecture

In the preceding output, we notice the following elements:

- `GeneralizedRCNNTransform` is a simple resize followed by a normalize transformation:

```
(transform): GeneralizedRCNNTransform(
    Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
    Resize(min_size=(800,), max_size=1333, mode='bilinear')
)
```

Figure 8.6: Transformation on input

- `BackboneWithFPN` is a neural network that transforms input into a feature map.
- `RegionProposalNetwork` generates the anchor boxes for the preceding feature map and predicts individual feature maps for classification and regression tasks:

```
(rpn): RegionProposalNetwork(
  (anchor_generator): AnchorGenerator()
  (head): RPNHead(
    (conv): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (cls_logits): Conv2d(256, 3, kernel_size=(1, 1), stride=(1, 1))
    (bbox_pred): Conv2d(256, 12, kernel_size=(1, 1), stride=(1, 1))
  )
)
```

Figure 8.7: RPN architecture

- `RoIHeads` takes the preceding maps, aligns them using ROI pooling, processes them, and returns classification probabilities for each proposal and the corresponding offsets:

```
(roi_heads): RoIHeads(
  (box_roi_pool): MultiScaleRoIAlign()
  (box_head): TwoMLPHead(
    (fc6): Linear(in_features=12544, out_features=1024, bias=True)
    (fc7): Linear(in_features=1024, out_features=1024, bias=True)
  )
  (box_predictor): FastRCNNPredictor(
    (cls_score): Linear(in_features=1024, out_features=2, bias=True)
    (bbox_pred): Linear(in_features=1024, out_features=8, bias=True)
  )
)
```

Figure 8.8: The roi_heads architecture

8. Define functions to train on batches of data and calculate loss values on the validation data:

```
# Defining training and validation functions
def train_batch(inputs, model, optimizer):
    model.train()
    input, targets = inputs
    input = list(image.to(device) for image in input)
    targets = [{k: v.to(device) for k, v in t.items()} for t in targets]
    optimizer.zero_grad()
    losses = model(input, targets)
    loss = sum(loss for loss in losses.values())
    loss.backward()
    optimizer.step()
    return loss, losses
@torch.no_grad()
def validate_batch(inputs, model):
    model.train()
#to obtain losses, model needs to be in train mode only
#Note that here we aren't defining the model's forward #method
#hence need to work per the way the model class is defined
    input, targets = inputs
    input = list(image.to(device) for image in input)
    targets = [{k: v.to(device) for k, v \
                in t.items()} for t in targets]
```

```
    optimizer.zero_grad()
    losses = model(input, targets)
    loss = sum(loss for loss in losses.values())
    return loss, losses
```

9. Train the model over increasing epochs:
    1. Define the model:

```
model = get_model().to(device)
optimizer = torch.optim.SGD(model.parameters(), lr=0.005,
                            momentum=0.9,weight_decay=0.0005)
n_epochs = 5
log = Report(n_epochs)
```

    2. Train the model and calculate the loss values on the training and test datasets:

```
for epoch in range(n_epochs):
    _n = len(train_loader)
    for ix, inputs in enumerate(train_loader):
        loss, losses = train_batch(inputs, model, optimizer)
        loc_loss, regr_loss, loss_objectness, \
            loss_rpn_box_reg = \
                [losses[k] for k in ['loss_classifier', \
                'loss_box_reg', 'loss_objectness', \
                'loss_rpn_box_reg']]
        pos = (epoch + (ix+1)/_n)
        log.record(pos, trn_loss=loss.item(),
                 trn_loc_loss=loc_loss.item(),
                 trn_regr_loss=regr_loss.item(),
                 trn_objectness_loss=loss_objectness.item(),
                trn_rpn_box_reg_loss=loss_rpn_box_reg.item(),
                 end='\r')
    _n = len(test_loader)
    for ix,inputs in enumerate(test_loader):
        loss, losses = validate_batch(inputs, model)
        loc_loss, regr_loss, loss_objectness, \
            loss_rpn_box_reg = \
                [losses[k] for k in ['loss_classifier', \
                'loss_box_reg', 'loss_objectness', \
                'loss_rpn_box_reg']]
        pos = (epoch + (ix+1)/_n)
        log.record(pos, val_loss=loss.item(),
                 val_loc_loss=loc_loss.item(),
                 val_regr_loss=regr_loss.item(),
                 val_objectness_loss=loss_objectness.item(),
                 val_rpn_box_reg_loss=loss_rpn_box_reg.item(), end='\r')
    if (epoch+1)%(n_epochs//5)==0: log.report_avgs(epoch+1)
```

10. Plot the variation of the various loss values over increasing epochs:

```
log.plot_epochs(['trn_loss','val_loss'])
```

This results in the following output:

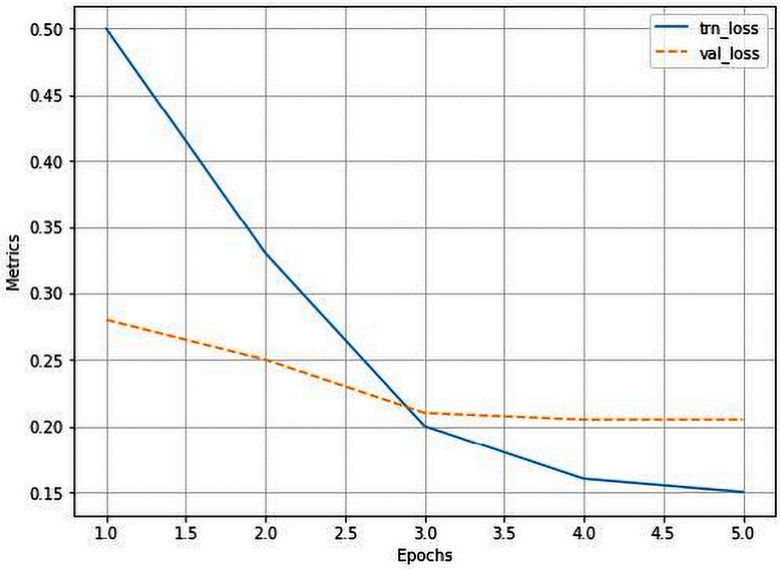

Figure 8.9: Training and validation loss over increasing epochs

11. Predict on a new image:
    1. The output of the trained model contains boxes, labels, and scores corresponding to classes. In the following code, we define a `decode_output` function that takes the model's output and provides the list of boxes, scores, and classes after non-max suppression:

```
from torchvision.ops import nms
def decode_output(output):
    'convert tensors to numpy arrays'
    bbs = output['boxes'].cpu().detach().numpy().astype(np.uint16)
    labels = np.array([target2label[i] for i in \
                output['labels'].cpu().detach().numpy()])
    confs = output['scores'].cpu().detach().numpy()
    ixs = nms(torch.tensor(bbs.astype(np.float32)),
                            torch.tensor(confs), 0.05)
    bbs, confs, labels = [tensor[ixs] for tensor in [bbs, confs, labels]]
    if len(ixs) == 1:
        bbs,confs,labels = [np.array([tensor]) for tensor \
                                in [bbs, confs, labels]]
    return bbs.tolist(), confs.tolist(), labels.tolist()
```

    2. Fetch the predictions of the boxes and classes on test images:

```
model.eval()
for ix, (images, targets) in enumerate(test_loader):
    if ix==3: break
    images = [im for im in images]
    outputs = model(images)
    for ix, output in enumerate(outputs):
        bbs, confs, labels = decode_output(output)
        info = [f'{l}@{c:.2f}' for l,c in zip(labels,confs)]
        show(images[ix].cpu().permute(1,2,0), bbs=bbs,
                                        texts=labels, sz=5)
```

The preceding code provides the following output:

Figure 8.10: Predicted bounding boxes and classes

Note that it now takes ~400 ms to generate predictions for one image, compared to 1.5 seconds with Fast R-CNN.

In this section, we have trained a Faster R-CNN model using the `fasterrcnn_resnet50_fpn` model class provided in the PyTorch `models` package. In the next section, we will learn about YOLO, a modern object detection algorithm that performs both object class detection and region correction in a single shot without the need to have a separate RPN.

Working details of YOLO

You Only Look Once (**YOLO**) and its variants are one of the prominent object detection algorithms. In this section, we will understand at a high level how YOLO works and the potential limitations of R-CNN-based object detection frameworks that YOLO overcomes.

First, let's understand the possible limitations of R-CNN-based detection algorithms. In Faster R-CNN, we slide over the image using anchor boxes and identify regions likely to contain an object, and then make the bounding box corrections. However, in the fully connected layer, where only the detected region's RoI pooling output is passed as input, in the case of regions that do not fully encompass the object (where the object is beyond the boundaries of the bounding box of region proposal), the network has to guess the real boundaries of the object, as it has not seen the full image (but has seen only the region proposal). YOLO comes in handy in such scenarios, as it looks at the whole image while predicting the bounding box corresponding to an image. Furthermore, Faster R-CNN is still slow, as we have two networks: the RPN and the final network that predicts classes and bounding boxes around objects.

Let's understand how YOLO overcomes the limitations of Faster R-CNN, both by looking at the whole image at once as well as by having a single network to make predictions. We will look at how data is prepared for YOLO through the following example.

First, we create a ground truth to train a model for a given image:

1. Let's consider an image with the given ground truth of bounding boxes in red:

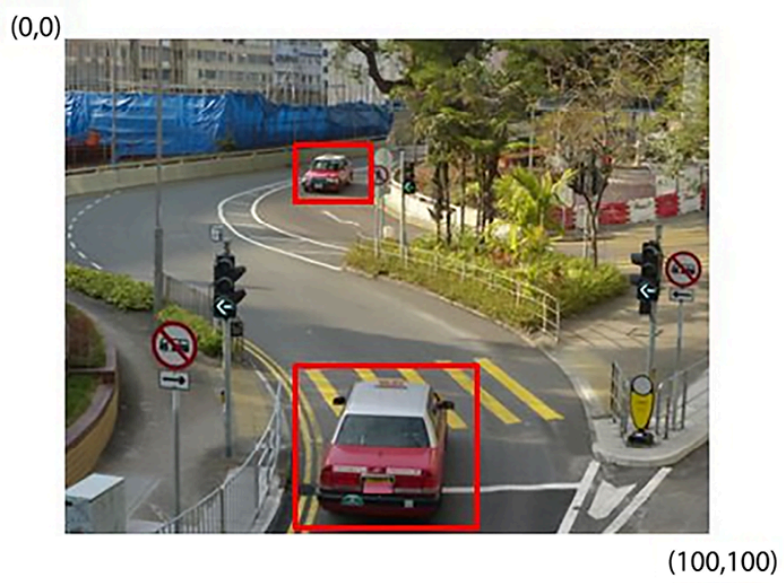

Figure 8.11: Input image with ground-truth bounding boxes

2. Divide the image into *N* x *N* grid cells – for now, let's say *N*=3:

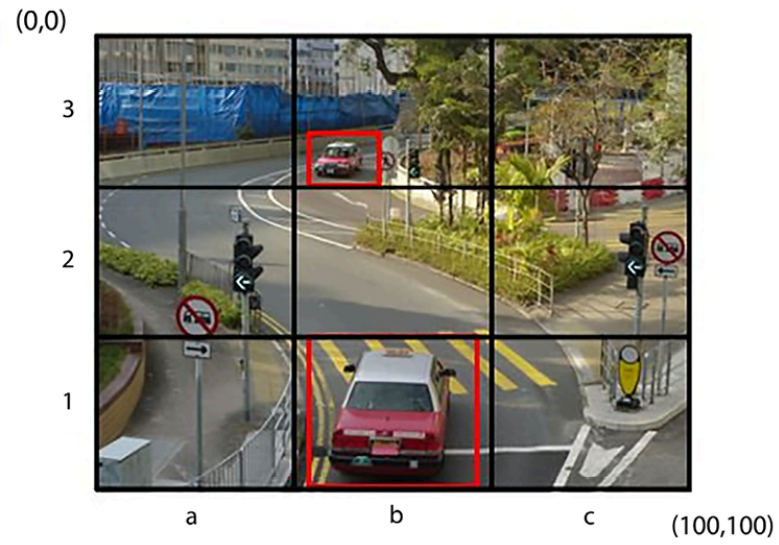

Figure 8.12: Dividing the input image into a 3 x 3 grid

3. Identify those grid cells that contain the center of at least one ground-truth bounding box. In our case, they are cells **b1** and **b3** of our 3 x 3 grid image.
4. The cell(s) where the middle point of the ground-truth bounding box falls is/are responsible for predicting the bounding box of the object. Let's create the ground truth corresponding to each cell.
5. The output ground truth corresponding to each cell is as follows:

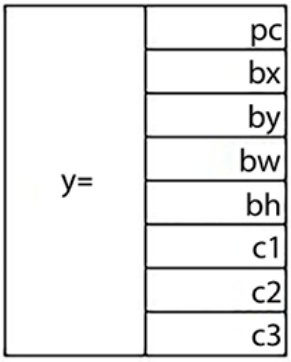

Figure 8.13: Ground truth representation

Here, **pc** (the objectness score) is the probability of the cell containing an object.

6. Let's understand how to calculate **bx**, **by**, **bw**, and **bh**. First, we consider the grid cell (let's consider the **b1** grid cell) as our universe, and normalize it to a scale between 0 and 1, as follows:

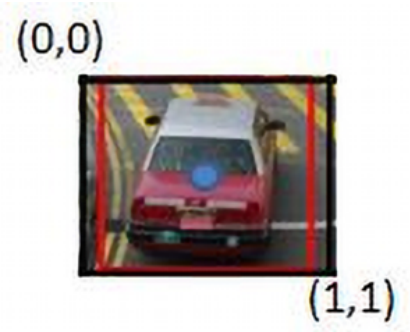

Figure 8.14: Step 1 of calculating bx, by, bw, and bh for each ground truth

bx and **by** are the locations of the midpoint of the ground-truth bounding box with respect to the image (of the grid cell), as defined previously. In our case, **bx** = 0.5, as the midpoint of the ground truth is at a distance of 0.5 units from the origin. Similarly, **by** = 0.5:

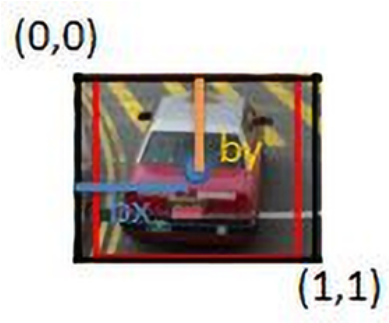

Figure 8.15: Calculating bx and by

So far, we have calculated offsets from the grid cell center to the ground truth center corresponding to the object in the image. Now, let's understand how **bw** and **bh** are calculated:

- **bw** is the ratio of the width of the bounding box with respect to the width of the grid cell.
- **bh** is the ratio of the height of the bounding box with respect to the height of the grid cell.

7. Next, we will predict the class corresponding to the grid cell. If we have three classes (`c1` – `truck` , `c2` – `car` , and `c3` – `bus`), we will predict the probability of the cell containing an object among any of the three classes. Note that we do not need a background class here, as **pc** corresponds to whether the grid cell contains an object.
8. Now that we understand how to represent the output layer of each cell, let's understand how we construct the output of our 3 x 3 grid cells:
    1. Let's consider the output of the grid cell **a3**:

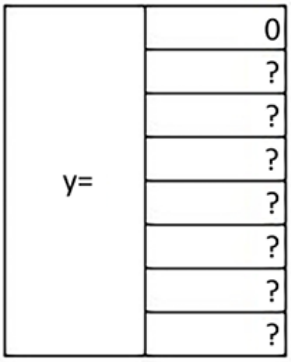

Figure 8.16: Calculating the ground truth corresponding to cell a3

The output of cell **a3** is as shown in the preceding screenshot. As the grid cell does not contain an object, the first output (**pc** – objectness score) is `0` and the remaining values do not matter as the cell does not contain the center of any ground-truth bounding boxes of an object.

    2. Let's consider the output corresponding to grid cell **b1**:

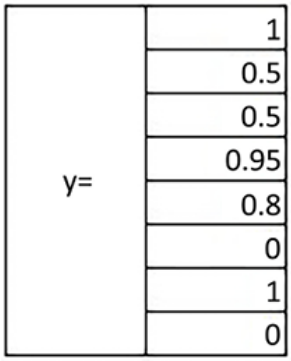

Figure 8.17: Ground truth values corresponding to cell b1

The preceding output is the way it is because the grid cell contains an object with the **bx**, **by**, **bw**, and **bh** values that were obtained in the same way as we went through earlier (in the bullet point before last), and finally, the class being `car` resulting in c2 being `1` while c1 and c3 are `0`.

Note that for each cell, we are able to fetch 8 outputs. Hence, for the 3 x 3 grid of cells, we fetch 3 x 3 x 8 outputs.

Let's look at the next steps:

1. Define a model where the input is an image and the output is 3 x 3 x 8 with the ground truth being as defined in the previous step:

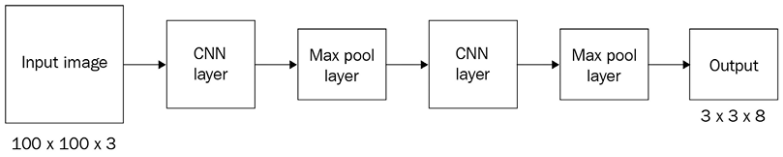

Figure 8.18: Sample model architecture

2. Define the ground truth by considering the anchor boxes.

So far, we have been building for a scenario where the expectation is that there is only one object within a grid cell. However, in reality, there can be scenarios where there are multiple objects within the same grid cell. This would result in creating ground truths that are incorrect. Let's understand this phenomenon through the following example image:

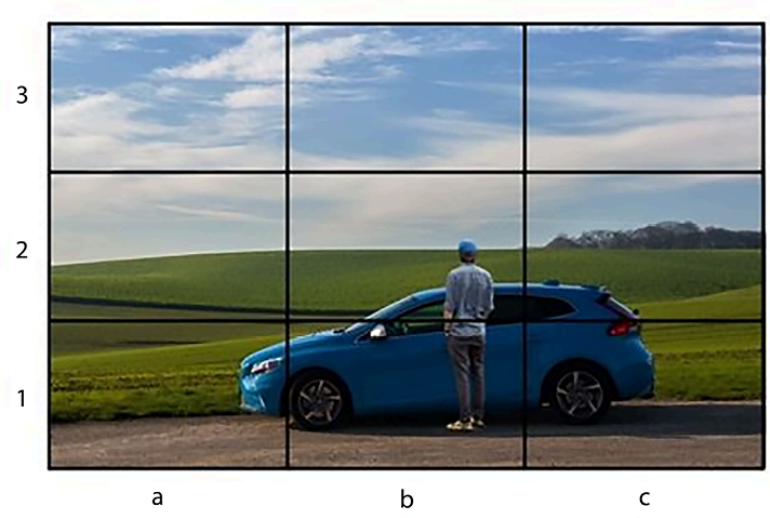

Figure 8.19: Scenario where there can be multiple objects in the same grid cell

In the preceding example, the midpoint of the ground-truth bounding boxes for both the car and the person fall in the same cell – cell **b1**.

One way to avoid such a scenario is by having a grid that has more rows and columns – for example, a 19 x 19 grid. However, there can still be a scenario where an increase in the number of grid cells does not help. Anchor boxes come in handy in such a scenario. Let's say we have two anchor boxes – one that has a greater height than width (corresponding to the person) and another that has a greater width than height (corresponding to the car):

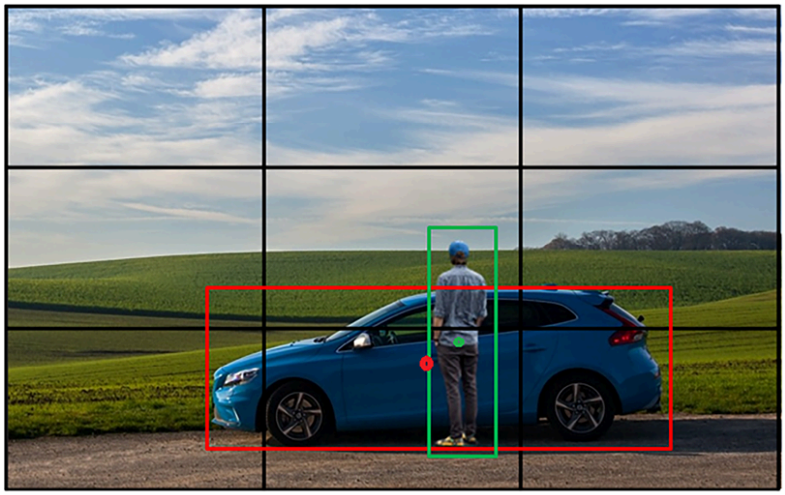

Figure 8.20: Leveraging anchor boxes

Typically, the anchor boxes would have the grid cell center as their centers. The output for each cell in a scenario where we have two anchor boxes is represented as a concatenation of the output expected of the two anchor boxes:

y=	pc
	bx
	by
	bh
	bw
	c1
	c2
	c3
	pc
	bx
	by
	bh
	bw
	c1
	c2
	c3

Figure 8.21: Ground truth representation when there are two anchor boxes

Here, **bx**, **by**, **bw**, and **bh** represent the offset from the anchor box (which is the universe in this scenario, as seen in the image instead of the grid cell).

From the preceding screenshot, we see we have an output that is 3 x 3 x 16, as we have two anchors. The expected output is of the shape *N* x *N* x `num_classes` x `num_anchor_boxes`, where *N* x *N* is the number of cells in the grid, `num_classes` is the number of classes in the dataset, and `num_anchor_boxes` is the number of anchor boxes.

3. Now we define the loss function to train the model.

When calculating the loss associated with the model, we need to ensure that we do not calculate the regression loss and classification loss when the objectness score is less than a certain threshold (this corresponds to the cells that do not contain an object).

Next, if the cell contains an object, we need to ensure that the classification across different classes is as accurate as possible.

Finally, if the cell contains an object, the bounding box offsets should be as close to expected as possible. However, since the offsets of width and height can be much higher when compared to the offset of the center (as offsets of the center range between 0 and 1, while the offsets of width and height need not), we give a lower weightage to offsets of width and height by fetching a square root value.

Calculate the loss of localization and classification as follows:

$$L_{loc} = \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^{B} 1_{ij}^{obj} [(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 + \left(\sqrt{w_i} - \sqrt{\hat{w}_i}\right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i}\right)^2]$$

$$L_{cls} = \sum_{i=0}^{S^2} \sum_{j=0}^{B} (1_{ij}^{obj} + \lambda_{noobj}(1 - 1_{ij}^{obj}))\left(C_{ij} - \hat{C}_{ij}\right)^2 + \sum_{i=0}^{S^2} \sum_{c \in C} 1_i^{obj} (p_i(c) - \hat{p}_i(c))^2$$

$$L = L_{loc} + L_{cls}$$

Here, we observe the following:

- is the weightage associated with regression loss
- represents whether the cell contains an object
- $\hat{p}_i(c)$ corresponds to the predicted class probability
- represents the objectness score

The overall loss is a sum of classification and regression loss values.

With this in place, we are now in a position to train a model to predict the bounding boxes around objects. However, for a stronger understanding of YOLO and its variants, we encourage you to go through the original paper at **https://arxiv.org/pdf/1506.02640**.

Now that we understand how YOLO predicts bounding boxes and classes of objects in a single shot, we will code it up in the next section.

Training YOLO on a custom dataset

Building on top of others' work is very important to becoming a successful practitioner in deep learning. For this implementation, we will use the official YOLOv4 implementation to identify the location of buses and trucks in images. We will clone the repository of the YOLO authors' own implementation and customize it to our needs in the following code.

To train the latest YOLO models, we strongly recommend you go through the following repos – **https://github.com/ultralytics/ultralytics** and **https://github.com/WongKinYiu/yolov7**.

We have provided the working implementation of YOLOv8 as `Training_YOLOv8.ipynb` within the `Chapter08` folder on GitHub at **https://bit.ly/mcvp-2e**.

Installing Darknet

First, pull the `darknet` repository from GitHub and compile it in the environment. The model is written in a separate language called Darknet, which is different from PyTorch. We will do so using the following code:

The following code can be found in the `Training_YOLO.ipynb` file in the `Chapter08` folder on GitHub at **https://bit.ly/mcvp-2e**.

1. Pull the Git repo:

```
!git clone https://github.com/AlexeyAB/darknet
%cd darknet
```

2. Reconfigure the `Makefile` file:

```
!sed -i 's/OPENCV=0/OPENCV=1/' Makefile
# In case you dont have a GPU, make sure to comment out the
# below 3 lines
!sed -i 's/GPU=0/GPU=1/' Makefile
!sed -i 's/CUDNN=0/CUDNN=1/' Makefile
!sed -i 's/CUDNN_HALF=0/CUDNN_HALF=1/' Makefile
```

`Makefile` is a configuration file needed for installing `darknet` in the environment (think of this process as similar to the selections you make when installing software on Windows). We are forcing `darknet` to be installed with the following flags: `OPENCV`, `GPU`, `CUDNN`, and `CUDNN_HALF`. These are all important optimizations to make the training faster.

Furthermore, in the preceding code, there is a curious function called `sed`, which stands for **stream editor**. It is a powerful Linux command that can modify information in text files directly from Command Prompt. Specifically, here we are using its search-and-replace function to replace `OPENCV=0` with `OPENCV=1`, and so on. The syntax to understand here is `sed 's/<search-string>/<replace-with>/'path/to/text/file`.

3. Compile the `darknet` source code:

```
!make
```

4. Install the `torch_snippets` package:

```
!pip install -q torch_snippets
```

5. Download and extract the dataset, and remove the ZIP file to save space:

```
!wget --quiet \
 https://www.dropbox.com/s/agmzwk95v96ihic/open-images-bus-trucks.tar.xz
!tar -xf open-images-bus-trucks.tar.xz
!rm open-images-bus-trucks.tar.xz
```

6. Fetch the pre-trained weights to make a sample prediction:

```
!wget --quiet\ https://github.com/AlexeyAB/darknet/releases/download/darknet_yolo_v3_optimal
```

7. Test whether the installation is successful by running the following command:

```
!./darknet detector test cfg/coco.data cfg/yolov4.cfg\ yolov4.weights
 data/person.jpg
```

This would make a prediction on `data/person.jpg` using the network built from `cfg/yolov4.cfg` and pre-trained weights – `yolov4.weights`. Furthermore, it fetches the classes from `cfg/coco.data`, which is what the pre-trained weights were trained on.

The preceding code results in predictions on the sample image (`data/person.jpg`), as follows:

Figure 8.22: Prediction on a sample image

Now that we have learned about installing `darknet`, in the next section, we will learn about creating ground truths for our custom dataset to leverage `darknet`.

Setting up the dataset format

YOLO uses a fixed format for training. Once we store the images and labels in the required format, we can train on the dataset with a single command. So, let's learn about the files and folder structure needed for YOLO to train.

There are three important steps:

1. Create a text file at `data/obj.names` containing the names of classes, one class per line, by running the following line (`%%writefile` is a magic command that creates a text file at `data/obj.names` with whatever content is present in the notebook cell):

```
%%writefile data/obj.names
bus
truck
```

2. Create a text file at `data/obj.data` describing the parameters in the dataset and the locations of text files containing the train and test image paths and the location of the file containing object names and the folder where you want to save trained models:

```
%%writefile data/obj.data
classes = 2
train = data/train.txt
valid = data/val.txt
names = data/obj.names
backup = backup/
```

The extensions for the preceding text files are not `.txt`. YOLO uses hardcoded names and folders to identify where data is. Also, the magic `%%writefile` Jupyter function creates a file with the content mentioned in a cell, as shown previously. Treat each `%%writefile ...` as a separate cell in Jupyter.

3. Move all images and ground-truth text files to the `data/obj` folder. We will copy images from the `bus-trucks` dataset to this folder along with the labels:

```
!mkdir -p data/obj
!cp -r open-images-bus-trucks/images/* data/obj/
!cp -r open-images-bus-trucks/yolo_labels/all/{train,val}.txt data/
!cp -r open-images-bus-trucks/yolo_labels/all/labels/*.txt data/obj/
```

Note that all the training and validation images are in the same `data/obj` folder. We also move a bunch of text files to the same folder. Each file that contains the ground truth for an image shares the same name as the image. For example, the folder might contain `1001.jpg` and

`1001.txt`, implying that the text file contains labels and bounding boxes for that image. If `data/train.txt` contains `1001.jpg` as one of its lines, then it is a training image. If it's present in `val.txt`, then it is a validation image.

The text file itself should contain information like so: `cls`, `xc`, `yc`, `w`, `h`, where `cls` is the class index of the object in the bounding box present at (`xc`, `yc`), which represents the centroid of the rectangle of width `w` and height `h`. Each of `xc`, `yc`, `w`, and `h` is a fraction of the image width and height. Store each object on a separate line.

For example, if an image of width (`800`) and height (`600`) contains one truck and one bus at centers (`500,300`) and (`100,400`) respectively, and has widths and heights of (`200,100`) and (`300,50`) respectively, then the text file would look as follows:

```
1 0.62 0.50 0.25 0.12
0 0.12 0.67 0.38 0.08
```

Now that we have created the data, let's configure the network architecture in the next section.

Configuring the architecture

YOLO comes with a long list of architectures. Some are large and some are small, to train on large or small datasets. Configurations can have different backbones. There are pre-trained configurations for standard datasets. Each configuration is a `.cfg` file present in the `cfgs` folder of the same GitHub repo that we cloned.

Each of them contains the architecture of the network as a text file (as opposed to how we were building it with the `nn.Module` class) along with a few hyperparameters, such as batch size and learning rate. We will take the smallest available architecture and configure it for our dataset:

```
# create a copy of existing configuration and modify it in place
!cp cfg/yolov4-tiny-custom.cfg cfg/yolov4-tiny-bus-trucks.cfg
# max_batches to 4000 (since the dataset is small enough)
!sed -i 's/max_batches = 500200/max_batches=4000/' cfg/yolov4-tiny-bus-trucks.cfg
# number of sub-batches per batch
!sed -i 's/subdivisions=1/subdivisions=16/' cfg/yolov4-tiny-bus-trucks.cfg
# number of batches after which learning rate is decayed
!sed -i 's/steps=400000,450000/steps=3200,3600/' cfg/yolov4-tiny-bus-trucks.cfg
# number of classes is 2 as opposed to 80
# (which is the number of COCO classes)
!sed -i 's/classes=80/classes=2/g' cfg/yolov4-tiny-bus-trucks.cfg
# in the classification and regression heads,
# change number of output convolution filters
# from 255 -> 21 and 57 -> 33, since we have fewer classes
# we don't need as many filters
!sed -i 's/filters=255/filters=21/g' cfg/yolov4-tiny-bus-trucks.cfg
!sed -i 's/filters=57/filters=33/g' cfg/yolov4-tiny-bus-trucks.cfg
```

This way, we have repurposed `yolov4-tiny` to be trainable on our dataset. The only remaining step is to load the pre-trained weights and train the model, which we will do in the next section.

Training and testing the model

We will get the weights from the following GitHub location and store them in `build/darknet/x64`:

```
!wget --quiet https://github.com/AlexeyAB/darknet/releases/download/darknet_yolo_v4_pre/yolov4
!cp yolov4-tiny.conv.29 build/darknet/x64/
```

Finally, we will train the model using the following line:

```
!./darknet detector train data/obj.data \
cfg/yolov4-tiny-bus-trucks.cfg yolov4-tiny.conv.29 -dont_show -mapLastAt
```

The `-dont_show` flag skips showing intermediate prediction images and `-mapLastAt` will periodically print the mean average precision on the validation data. The whole of the training might take 1 or 2 hours with GPU. The weights are periodically stored in a backup folder and can be used after training for predictions such as the following code, which makes predictions on a new image:

```
!pip install torch_snippets
from torch_snippets import Glob, stem, show, read
# upload your own images to a folder
image_paths = Glob('images-of-trucks-and-busses')
for f in image_paths:
    !./darknet detector test \
    data/obj.data cfg/yolov4-tiny-bus-trucks.cfg\
    backup/yolov4-tiny-bus-trucks_4000.weights {f}
    !mv predictions.jpg {stem(f)}_pred.jpg
for i in Glob('*_pred.jpg'):
    show(read(i, 1), sz=20)
```

The preceding code results in the following output:

Figure 8.23: Predicted bounding box and class on input images

Now that we have learned about leveraging YOLO to perform object detection on our custom dataset, in the next section, we will learn about another object detection technique – **Single-Shot Detector** (**SSD**) to perform object detection.

Working details of SSD

So far, we have seen a scenario where we made predictions after gradually convolving and pooling the output from the previous layer. However,

we know that different layers have different receptive fields to the original image. For example, the initial layers have a smaller receptive field when compared to the final layers, which have a larger receptive field. Here, we will learn how SSD leverages this phenomenon to come up with a prediction of bounding boxes for images.

The workings behind how SSD helps overcome the issue of detecting objects with different scales is as follows:

1. We leverage the pre-trained VGG network and extend it with a few additional layers until we obtain a 1 x 1 block.
2. Instead of leveraging only the final layer for bounding box and class predictions, we will leverage all of the last few layers to make class and bounding box predictions.
3. In place of anchor boxes, we will come up with default boxes that have a specific set of scale and aspect ratios.
4. Each of the default boxes should predict the object and bounding box offset, just like how anchor boxes are expected to predict classes and offsets in YOLO.

Now that we understand the main ways in which SSD differs from YOLO (which is that default boxes in SSD replace anchor boxes in YOLO and multiple layers are connected to the final layer in SSD, instead of gradual convolution pooling in YOLO), let's learn about the following:

- The network architecture of SSD
- How to leverage different layers for bounding box and class predictions
- How to assign scale and aspect ratios for default boxes in different layers

The network architecture of SSD is as follows:

Figure 8.24: SSD workflow

As you can see in the preceding diagram, we are taking an image of size 300 x 300 x 3 and passing it through a pre-trained VGG-16 network to obtain the `conv5_3` layer's output. Furthermore, we are extending the network by adding a few more convolutions to the `conv5_3` output.

Next, we obtain a bounding box offset and class prediction for each cell and each default box (more on default boxes in the next section; for now, let's imagine that this is similar to an anchor box). The total number of predictions coming from the `conv5_3` output is 38 x 38 x 4, where 38 x 38 is the output shape of the `conv5_3` layer and 4 is the number of default boxes operating on the `conv5_3` layer. Similarly, the total number of detections across the network is as follows:

Layer	Number of detections per class
`conv5_3`	38 x 38 x 4 = 5,776
`FC6`	19 x 19 x 6 = 2,166
`conv8_2`	10 x 10 x 6 = 600
`conv9_2`	5 x 5 x 6 = 150
`conv10_2`	3 x 3 x 4 = 36
`conv11_2`	1 x 1 x 4 = 4
Total detections	**8,732**

Table 8.1: Number of detections per class

Note that certain layers have a larger number of default boxes (6 and not 4) when compared to other layers in the architecture described in the original paper.

Now, let's learn about the different scales and aspect ratios of default boxes. We will start with scales and then proceed to aspect ratios.

Let's imagine a scenario where the minimum scale of an object is 20% of the height and 20% of the width of an image, and the maximum scale of the object is 90% of the height and 90% of the width. In such a scenario, we gradually increase scale across layers (as we proceed toward later layers, the image size shrinks considerably), as follows:

Figure 8.25: Scale of box with respect to size of object across layers

The formula that enables the gradual scaling of the image is as follows:

With that understanding of how to calculate scale across layers, let's learn about coming up with boxes of different aspect ratios. The possible aspect ratios are as follows:

The centers of the box for different layers are as follows:

Here, *i* and *j* together represent a cell in layer *l*. On the other hand, the width and height corresponding to different aspect ratios are calculated as follows:

Note that we were considering four boxes in certain layers and six boxes in another layer. If we want to have four boxes, we remove the `{3,1/3}` aspect ratios, else we consider all of the six possible boxes (five boxes with the same scale and one box with a different scale). Let's see how we obtain the sixth box:

With that, we have all the possible boxes. Let's understand how we prepare the training dataset next.

The default boxes that have an IoU greater than a threshold (say, `0.5`) with the ground truth are considered positive matches, and the rest are negative matches. In the output of SSD, we predict the probability of the box belonging to a class (where the 0^{th} class represents the background) and also the offset of the ground truth with respect to the default box.

Finally, we train the model by optimizing the following loss values:

- **Classification loss**: This is represented using the following equation:

In the preceding equation, *pos* represents the few default boxes that have a high overlap with the ground truth, while *neg* represents the misclassified boxes that were predicting a class but in fact did not contain an object. Finally, we ensure that the *pos:neg* ratio is at most 1:3, as if we do not perform this sampling, we would have a dominance of background class boxes.

- **Localization loss:** For localization, we consider the loss values only when the objectness score is greater than a certain threshold. The localization loss is calculated as follows:

Here, *t* is the predicted offset and *d* is the actual offset.

For an in-depth discussion on the SSD workflow, you can refer to https://arxiv.org/abs/1512.02325.

Now that we understand how to train SSD, let's use it for our bus versus truck object detection exercise in the next section. The core utility functions for this section are present in the GitHub repo: https://github.com/sizhky/ssd-utils/. Let's learn about them one by one before starting the training process.

Components in SSD code

There are three files in the GitHub repo. Let's dig into them a little and understand them before training. **Note that this section is not part of the training process, but is instead for understanding the imports used during training.**

We are importing the `SSD300` and `MultiBoxLoss` classes from the `model.py` file in the GitHub repository. Let's learn about both of them.

SSD300

When you look at the `SSD300` function definition, it is evident that the model comprises three submodules:

```
class SSD300(nn.Module):
    ...
    def __init__(self, n_classes, device):
        ...
        self.base = VGGBase()
        self.aux_convs = AuxiliaryConvolutions()
        self.pred_convs = PredictionConvolutions(n_classes)
        ...
```

We send the input to `VGGBase` first, which returns two feature vectors of dimensions `(N, 512, 38, 38)` and `(N, 1024, 19, 19)`. The second output is going to be the input for `AuxiliaryConvolutions`, which returns more feature maps of dimensions `(N, 512, 10, 10)`, `(N, 256, 5, 5)`, `(N, 256, 3, 3)`, and `(N, 256, 1, 1)`. Finally, the first output from `VGGBase` and these four feature maps are sent to `PredictionConvolutions`, which returns 8,732 anchor boxes, as we discussed previously.

The other key aspect of the `SSD300` class is the `create_prior_boxes` method. For every feature map, there are three items associated with it: the size of the grid, the scale to shrink the grid cell by (this is the base anchor box for this feature map), and the aspect ratios for all anchors in a cell. Using these three configurations, the code uses a triple `for` loop and creates a list of `(cx, cy, w, h)` for all 8,732 anchor boxes.

Finally, the `detect_objects` method takes tensors of classification and regression values (of the predicted anchor boxes) and converts them to

actual bounding box coordinates.

MultiBoxLoss

As humans, we are only worried about a handful of bounding boxes. But for the way SSD works, we need to compare 8,732 bounding boxes from several feature maps and predict whether an anchor box contains valuable information or not. We assign this loss computation task to `MultiBoxLoss`.

The input for the forward method is the anchor box predictions from the model and the ground-truth bounding boxes.

First, we convert the ground-truth boxes into a list of 8,732 anchor boxes by comparing each anchor from the model with the bounding box. If the IoU is high enough, that particular anchor box will have non-zero regression coordinates and associate an object as the ground truth for classification. Naturally, most of the computed anchor boxes will have their associated class as `background` because their IoU with the actual bounding box will be tiny or, in quite a few cases, `0`.

Once the ground truths are converted to these 8,732 anchor box regression and classification tensors, it is easy to compare them with the model's predictions since the shapes are now the same. We perform `MSE-Loss` on the regression tensor and `CrossEntropy-Loss` on the localization tensor and add them up to be returned as the final loss.

Training SSD on a custom dataset

In the following code, we will train the SSD algorithm to detect the bounding boxes around objects present in images. We will use the truck versus bus object detection task we have been working on:

Find the following code in the `Training_SSD.ipynb` file in the `Chapter08` folder on GitHub at **https://bit.ly/mcvp-2e**. The code contains URLs to download data from and is moderately lengthy. We strongly recommend executing the notebook in GitHub to reproduce the results while following the steps and explanations of various code components from the text.

1. Download the image dataset and clone the Git repository hosting the code for the model and the other utilities for processing the data:

```
import os
if not os.path.exists('open-images-bus-trucks'):
    !pip install -q torch_snippets
    !wget --quiet https://www.dropbox.com/s/agmzwk95v96ihic/\
    open-images-bus-trucks.tar.xz
    !tar -xf open-images-bus-trucks.tar.xz
    !rm open-images-bus-trucks.tar.xz
```

```
    !git clone https://github.com/sizhky/ssd-utils/
%cd ssd-utils
```

2. Preprocess the data, just like we did in the *Training Faster R-CNN on a custom dataset* section:

```
from torch_snippets import *
DATA_ROOT = '../open-images-bus-trucks/'
IMAGE_ROOT = f'{DATA_ROOT}/images'
DF_RAW = pd.read_csv(f'{DATA_ROOT}/df.csv')
df = DF_RAW.copy()
df = df[df['ImageID'].isin(df['ImageID'].unique().tolist())]
label2target = {l:t+1 for t,l in enumerate(DF_RAW['LabelName'].unique())}
label2target['background'] = 0
target2label = {t:l for l,t in label2target.items()}
background_class = label2target['background']
num_classes = len(label2target)
device = 'cuda' if torch.cuda.is_available() else 'cpu'
```

3. Prepare a dataset class, just like we did in the *Training Faster R-CNN on a custom dataset* section:

```
import collections, os, torch
from PIL import Image
from torchvision import transforms
normalize = transforms.Normalize(
                mean=[0.485, 0.456, 0.406],
                std=[0.229, 0.224, 0.225]
            )
denormalize = transforms.Normalize(
                mean=[-0.485/0.229,-0.456/0.224,-0.406/0.255],
                std=[1/0.229, 1/0.224, 1/0.255]
            )
def preprocess_image(img):
    img = torch.tensor(img).permute(2,0,1)
    img = normalize(img)
    return img.to(device).float()
class OpenDataset(torch.utils.data.Dataset):
    w, h = 300, 300
    def __init__(self, df, image_dir=IMAGE_ROOT):
        self.image_dir = image_dir
        self.files = glob.glob(self.image_dir+'/*')
        self.df = df
        self.image_infos = df.ImageID.unique()
        logger.info(f'{len(self)} items loaded')

    def __getitem__(self, ix):
        # load images and masks
        image_id = self.image_infos[ix]
        img_path = find(image_id, self.files)
        img = Image.open(img_path).convert("RGB")
        img = np.array(img.resize((self.w, self.h),
                        resample=Image.BILINEAR))/255.
        data = df[df['ImageID'] == image_id]
        labels = data['LabelName'].values.tolist()
        data = data[['XMin','YMin','XMax','YMax']].values
        data[:,[0,2]] *= self.w
```

```
            data[:,[1,3]] *= self.h
            boxes = data.astype(np.uint32).tolist() # convert to
            # absolute coordinates
            return img, boxes, labels
        def collate_fn(self, batch):
            images, boxes, labels = [], [], []
            for item in batch:
                img, image_boxes, image_labels = item
                img = preprocess_image(img)[None]
                images.append(img)
                boxes.append(torch.tensor( \
                           image_boxes).float().to(device)/300.)
                labels.append(torch.tensor([label2target[c] \
                      for c in image_labels]).long().to(device))
            images = torch.cat(images).to(device)
            return images, boxes, labels
        def __len__(self):
            return len(self.image_infos)
```

4. Prepare the training and test datasets and the dataloaders:

```
from sklearn.model_selection import train_test_split
trn_ids, val_ids = train_test_split(df.ImageID.unique(),
                                    test_size=0.1, random_state=99)
trn_df, val_df = df[df['ImageID'].isin(trn_ids)], \
                 df[df['ImageID'].isin(val_ids)]
train_ds = OpenDataset(trn_df)
test_ds = OpenDataset(val_df)
train_loader = DataLoader(train_ds, batch_size=4,
                          collate_fn=train_ds.collate_fn,
                          drop_last=True)
test_loader = DataLoader(test_ds, batch_size=4,
                         collate_fn=test_ds.collate_fn,
                         drop_last=True)
```

5. Define functions to train on a batch of data and calculate the accuracy and loss values on the validation data:

```
def train_batch(inputs, model, criterion, optimizer):
    model.train()
    N = len(train_loader)
    images, boxes, labels = inputs
    _regr, _clss = model(images)
    loss = criterion(_regr, _clss, boxes, labels)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    return loss
@torch.no_grad()
def validate_batch(inputs, model, criterion):
    model.eval()
    images, boxes, labels = inputs
    _regr, _clss = model(images)
    loss = criterion(_regr, _clss, boxes, labels)
    return loss
```

6. Import the model:

```
from model import SSD300, MultiBoxLoss
from detect import *
```

7. Initialize the model, optimizer, and loss function:

```
n_epochs = 5
model = SSD300(num_classes, device)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4, weight_decay=1e-5)
criterion = MultiBoxLoss(priors_cxcy=model.priors_cxcy, device=device)
log = Report(n_epochs=n_epochs)
logs_to_print = 5
```

8. Train the model over increasing epochs:

```
for epoch in range(n_epochs):
    _n = len(train_loader)
    for ix, inputs in enumerate(train_loader):
        loss = train_batch(inputs, model, criterion, optimizer)
        pos = (epoch + (ix+1)/_n)
        log.record(pos, trn_loss=loss.item(), end='\r')
    _n = len(test_loader)
    for ix,inputs in enumerate(test_loader):
        loss = validate_batch(inputs, model, criterion)
        pos = (epoch + (ix+1)/_n)
        log.record(pos, val_loss=loss.item(), end='\r')
```

The variation of training and test loss values over epochs is as follows:

Figure 8.26: Training and validation loss over increasing epochs

9. Fetch a prediction on a new image (fetch a random image):

```
image_paths = Glob(f'{DATA_ROOT}/images/*')
image_id = choose(test_ds.image_infos)
img_path = find(image_id, test_ds.files)
original_image = Image.open(img_path, mode='r')
original_image = original_image.convert('RGB')
```

10. Fetch the bounding box, label, and score corresponding to the objects present in the image:

```
bbs, labels, scores = detect(original_image, model,
                             min_score=0.9, max_overlap=0.5,
                             top_k=200, device=device)
```

11. Overlay the obtained output on the image:

```
labels = [target2label[c.item()] for c in labels]
label_with_conf = [f'{l} @ {s:.2f}' for l,s in zip(labels,scores)]
print(bbs, label_with_conf)
show(original_image, bbs=bbs,
     texts=label_with_conf, text_sz=10)
```

The preceding code fetches a sample of outputs as follows (one image for each iteration of execution):

Figure 8.27: Predicted bounding box and class on input images

From this, we can see that we can detect objects in the image reasonably accurately.

Summary

In this chapter, we have learned about the working details of modern object detection algorithms: Faster R-CNN, YOLO, and SSD. We learned how they overcome the limitation of having two separate models – one for fetching region proposals and the other for fetching class and bounding box offsets on region proposals. Furthermore, we implemented Faster R-CNN using PyTorch, YOLO using `darknet`, and SSD from scratch.

In the next chapter, we will learn about image segmentation, which goes one step beyond object localization by identifying the pixels that correspond to an object. Furthermore, in *Chapter 10*, *Applications of Object Detection and Segmentation*, we will learn about the Detectron2 framework, which helps in not only detecting objects but also segmenting them in a single shot.

Questions

1. Why is Faster R-CNN faster when compared to Fast R-CNN?
2. How are YOLO and SSD faster when compared to Faster R-CNN?
3. What makes YOLO and SSD single-shot detector algorithms?
4. What is the difference between the objectness score and class score?

Learn more on Discord

Join our community's Discord space for discussions with the authors and other readers:

https://packt.link/modcv